Compte-rendu du TP 3 : Connexion et Refactorisation de l'API Blog avec MongoDB Atlas et Mongoose

Réalisé par : **JOSEPH MUKUBU KAPOYA**
5ème Datascience et IA, Groupe 1

14 octobre 2026

Introduction

Ce troisième atelier pratique marque le passage d'une persistance volatile en mémoire à une véritable persistance de données distribuée et sécurisée grâce à l'**ODM Mongoose** et à la base de données **MongoDB Atlas**.

Les objectifs pédagogiques et techniques de ce TP sont :

- Sécuriser les secrets de configuration de l'application via des variables d'environnement (`.env` et bibliothèque `dotenv`).
- Centraliser et fiabiliser la connexion asynchrone à la base distante dans `src/config/db.js`.
- Définir des schémas et modèles stricts (`User` et `Article`) intégrant des validations automatiques (*regex*, types, contraintes numériques, énumérations, `timestamps: true`).
- Établir des relations référentielles NoSQL (`ObjectId` avec `ref: 'User'`) et exploiter la méthode `.populate()` pour récupérer les données jointes de l'auteur.
- Démontrer la puissance du découplage **SoC** : remplacement complet du moteur de persistance sans modifier le routeur.
- Traiter les contraintes d'intégrité relationnelle (interdiction de suppression d'utilisateurs avec articles) et gérer professionnellement les erreurs de format d'identifiant (`CastError`).

1 Réponse rédigée et argumentée à l'encadré « Prédisez »

Problématique : Découplage architectural et substitution de persistance

Question : *Pourquoi le passage d'un stockage en mémoire à une base de données MongoDB avec Mongoose n'a-t-il nécessité strictement aucune modification dans les routeurs (`routes/articleRoutes` et `routes/userRoutes.js`) ?*

Réponse argumentée :

Ce résultat découle directement de l'application rigoureuse du patron architectural **SoC (Separation of Concerns)** mis en place lors du TP 2.

1. **Étanchéité des responsabilités :** Le rôle unique d'un routeur Express est de faire correspondre un verbe HTTP (`GET`, `POST`, `PUT`, `DELETE`) et un chemin d'URL à un gestionnaire (*handler*) du contrôleur. Le routeur ignore totalement la provenance ou la méthode de stockage des données (qu'il s'agisse d'un tableau en RAM, d'un fichier JSON, ou d'une base MongoDB Atlas distante).

2. **Stabilité du contrat d'interface :** La signature des fonctions de contrôleur (`(req, res) => { ... }`) reste inchangée, même lorsqu'elles deviennent asynchrones (`async/await`). Le contrôleur encapsule toute la logique de requêtage Mongoose et renvoie des réponses HTTP au format JSON.
3. **Invariance pour le client :** Pour le consommateur externe (Postman, application front-end React), le contrat d'API (routes, codes HTTP, structure des payloads) est préservé, offrant ainsi une transition transparente sans aucune régression.

2 Captures d'Écran des Tests Postman

2.1 Création d'un utilisateur valide

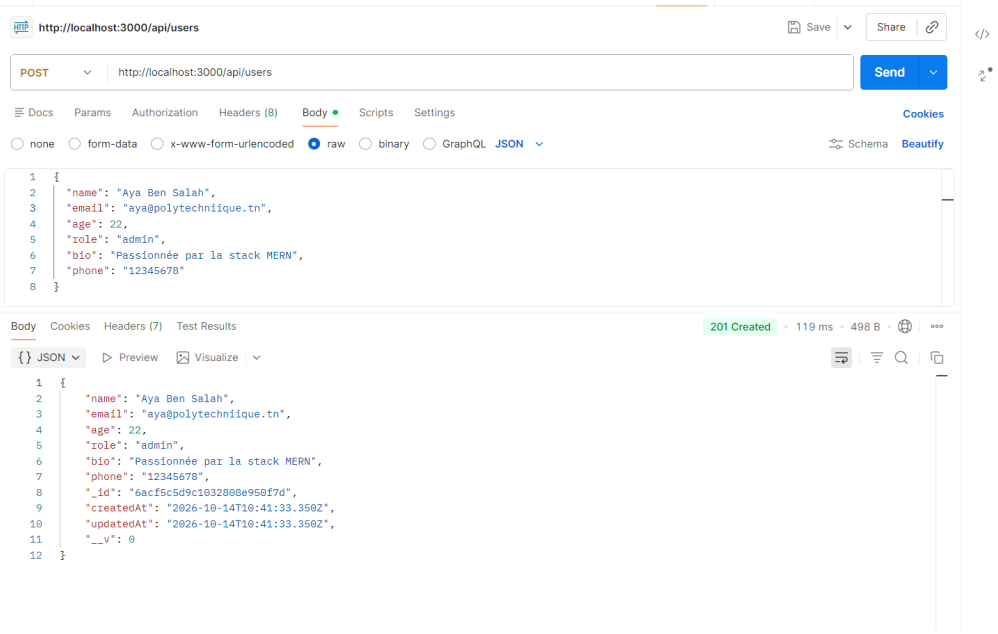


FIGURE 1 – POST /api/users – 201 Created (création utilisateur valide)

Description : Création d'un utilisateur **Aya Ben Salah** avec les champs `name`, `email`, `age`, `role`, `bio` et `phone`. Le serveur répond **201 Created** et retourne l'objet persisté en base avec son `_id` MongoDB et les horodatages `createdAt` / `updatedAt` générés automatiquement par Mongoose (`timestamps: true`).

2.2 Création d'un article avec authorId invalide

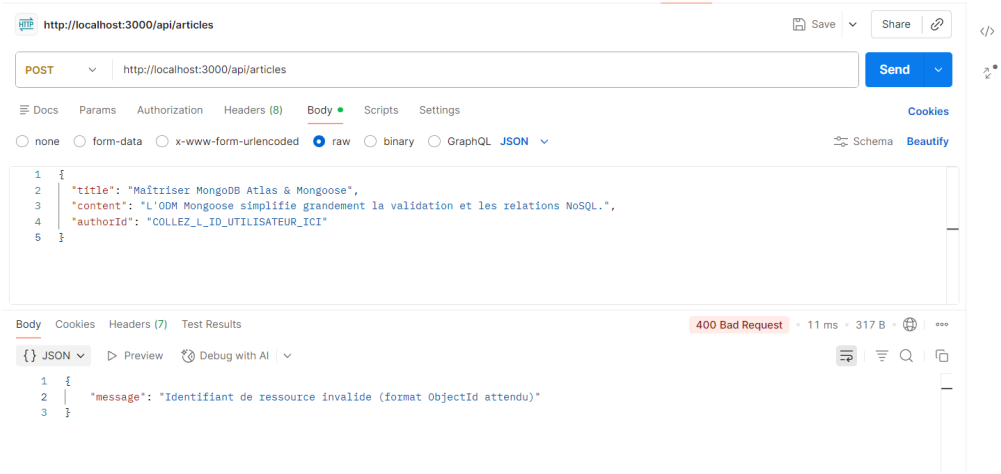


FIGURE 2 – POST /api/articles – 400 Bad Request (authorId invalide – CastError)

Description : Tentative de création d'un article avec la chaîne littérale `"COLLEZ_L_ID_UTILISATEUR_ICI"` comme `authorId`. Ce n'est pas un `ObjectId` valide : Mongoose lève une `CastError` interceptée par le bloc `catch`, qui renvoie **400 Bad Request** avec le message `"Identifiant de ressource invalide (format ObjectId attendu)"`.

2.3 Consultation des articles avec auteur peuplé

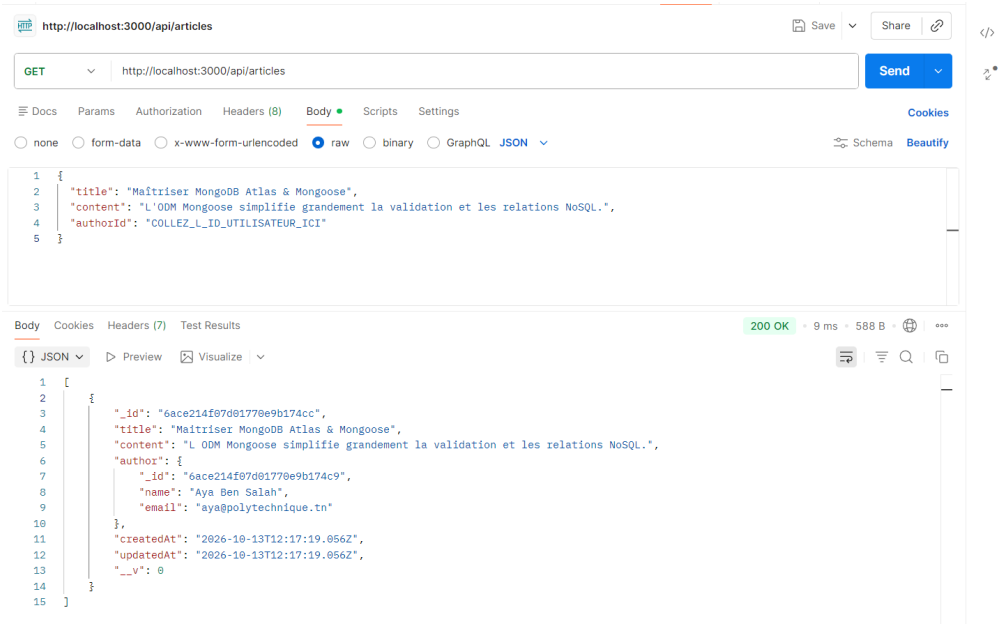


FIGURE 3 – GET /api/articles – 200 OK (populate auteur)

Description : Récupération de tous les articles. Grâce à `.populate('author', 'name email')`, le champ `author` n'est plus un simple `ObjectId` mais l'objet complet `{_id, name, email}` de l'auteur. On vérifie ici que l'article *"Maîtriser MongoDB Atlas & Mongoose"* est bien lié à **Aya Ben Salah** (`aya@polytechnique.tn`).

2.4 Tentative de doublon d'email

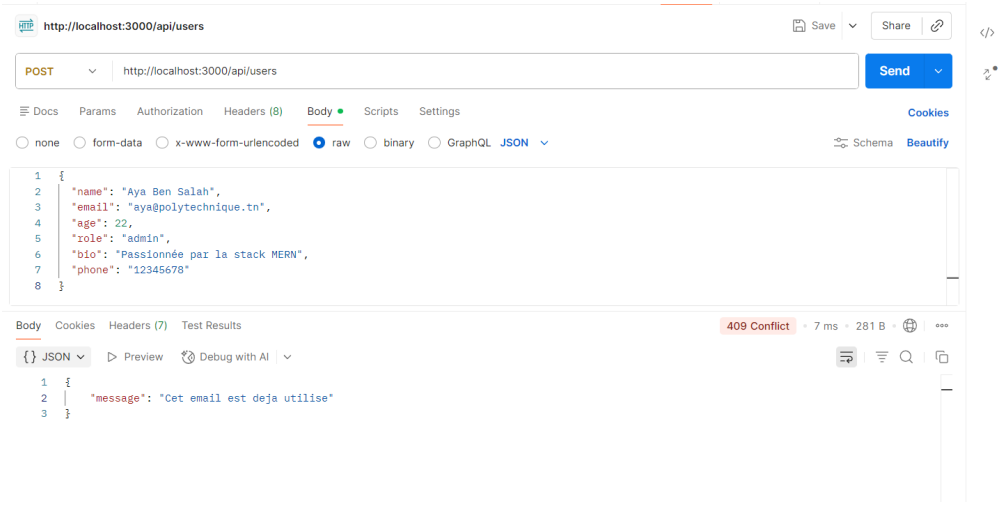


FIGURE 4 – POST /api/users – 409 Conflict (email déjà utilisé)

Description : Tentative de recréation du même utilisateur avec l'adresse `aya@polytechnique.tn` déjà présente en base. Le contrôleur détecte le doublon via `User.findOne({email})` et renvoie **409 Conflict** avec le message `"Cet email est deja utilise"`, sans déclencher d'erreur 500 ni de doublons en base.

2.5 Gestion des identifiants invalides – Exercice 2

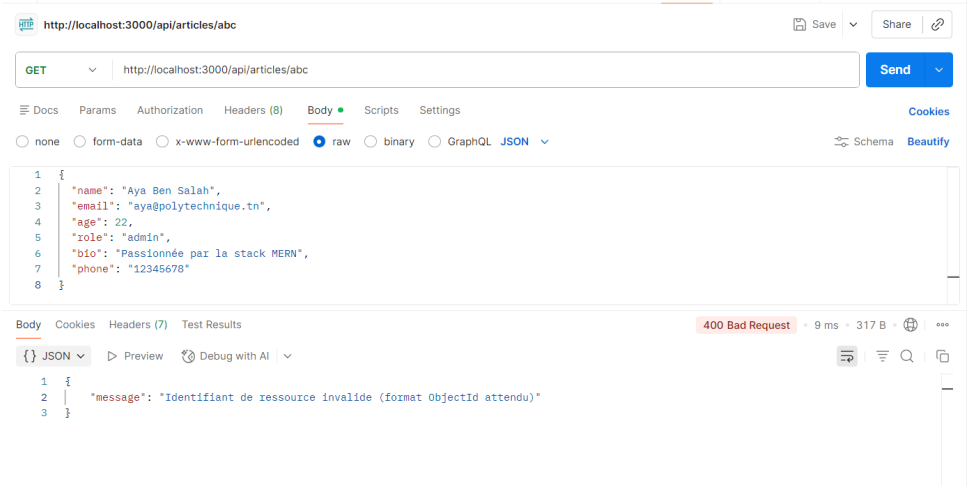


FIGURE 5 – GET /api/articles/abc – 400 Bad Request (CastError ObjectId)

Description : Appel sur `GET /api/articles/abc` avec un identifiant non hexadécimal. Mongoose tente de convertir `"abc"` en `ObjectId` et lève une `CastError`. Le bloc `catch` l'intercepte et retourne **400 Bad Request** avec `"Identifiant de ressource invalide (format ObjectId attendu)"` au lieu d'une erreur **500 Internal Server Error**.

2.6 Intégrité relationnelle : suppression bloquée – Exercice 1

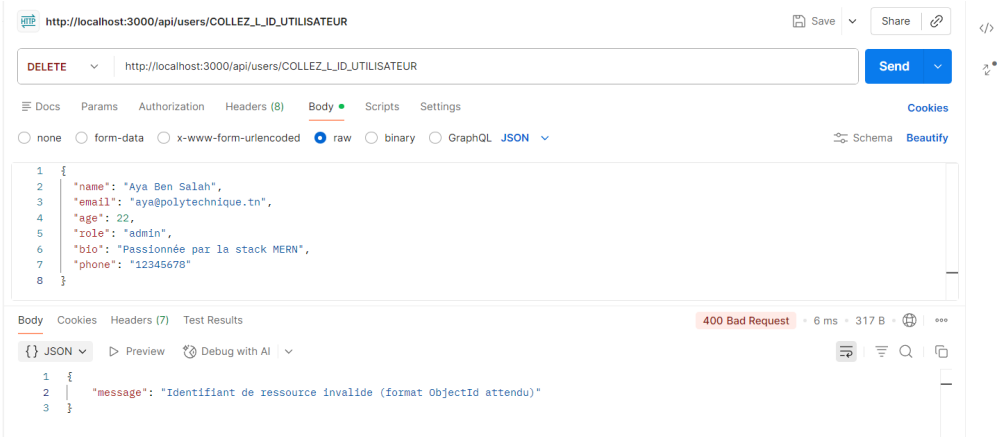


FIGURE 6 – DELETE /api/users/ :id – 400 Bad Request (identifiant ObjectId invalide)

Description : Tentative de suppression avec l'identifiant littéral `"COLLEZ_L_ID_UTILISATEUR"` (non ObjectId valide). Le serveur renvoie **400 Bad Request** avec le message `"Identifiant de ressource invalide (format ObjectId attendu)"`. Dans un scénario réel avec un ObjectId valide appartenant à un utilisateur ayant des articles, `deleteUser` vérifie d'abord `Article.findOne({author: id})` et bloque la suppression pour préserver l'intégrité référentielle.

3 Code Source Intégral du Projet

3.1 Configuration de l'environnement (.env)

```
PORT=3000
MONGODB_URI=mongodb+srv://admin_mern:PolyMern2026!@cluster0.xxxxx.mongodb.net/blogDB?retryWrites=true&w=majority
```

3.2 Centralisation de la connexion : src/config/db.js

```
const mongoose = require('mongoose');

async function connectDB(uri) {
  if (!uri) {
    throw new Error("L'URI MongoDB est manquant dans les variables d'environnement (.env)");
  }
  await mongoose.connect(uri, {
    autoIndex: true // Genere automatiquement les index uniques en developpement
  });
  console.log('Connexion a MongoDB Atlas reussie avec succes !');
}

module.exports = connectDB;
```

3.3 Point d'entrée du serveur : src/server.js

```
const express = require('express');
const dotenv = require('dotenv');
const connectDB = require('./config/db');
const articleRoutes = require('./routes/articleRoutes');
const userRoutes = require('./routes/userRoutes');

// Chargement des variables d'environnement (.env)
dotenv.config();

const app = express();
const PORT = process.env.PORT || 3000;

// Middleware pour parser le corps des requetes en JSON
app.use(express.json());

// Montage des routeurs modulaires
app.use('/api/articles', articleRoutes);
app.use('/api/users', userRoutes);

// Connexion asynchrone a MongoDB Atlas puis demarrage de l'ecoute
(async () => {
  try {
    // 1. Etablissement de la connexion distante
    await connectDB(process.env.MONGODB_URI);

    // 2. Demarrage de l'ecoute du serveur HTTP
    app.listen(PORT, () => {
      console.log(`Serveur MERN en ecoute sur http://localhost:${PORT}`);
```

```
    });
  } catch (err) {
    console.error('Erreur critique de demarrage du serveur :', err.message);
    process.exit(1);
  }
})();
```

3.4 Modèle Utilisateur avec contraintes avancées : src/models/userModel.js

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Le nom est obligatoire'],
      trim: true
    },
    email: {
      type: String,
      required: [true, "L'email est obligatoire"],
      unique: true,
      lowercase: true,
      trim: true,
      match: [/^[^\s@]+@[^\s@]+\.[^\s@]+$/, "Format d'email invalide (ex:
          aya@polytechnique.tn)"]
    },
    age: {
      type: Number,
      min: [4, "L'age minimum autorise est de 4 ans"]
    },
    role: {
      type: String,
      enum: {
        values: ['user', 'admin'],
        message: "{VALUE} n'est pas un role autorise"
      },
      default: 'user'
    },
    // Exercice 1 : Champs enrichis avec validations avancees
    bio: {
      type: String,
      maxlength: [200, 'La biographie ne peut pas depasser 200 caracteres']
    },
    phone: {
      type: String,
      match: [/^\d{8}$/, 'Numero de 8 chiffres requis']
    }
  },
  { timestamps: true } // Injection automatique de createdAt et updatedAt
);

module.exports = mongoose.model('User', userSchema);
```

3.5 Modèle Article avec relation NoSQL : src/models/articleModel.js

```
const mongoose = require('mongoose');
```

```
const articleSchema = new mongoose.Schema(
  {
    title: {
      type: String,
      required: [true, 'Le titre est obligatoire'],
      trim: true
    },
    content: {
      type: String,
      required: [true, 'Le contenu est obligatoire']
    },
    author: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: [true, "L'auteur (authorId) est obligatoire"]
    }
  },
  { timestamps: true }
);

module.exports = mongoose.model('Article', articleSchema);
```

3.6 Contrôleur Utilisateurs : src/controllers/userController.js

```
const User = require('../models/userModel');
const Article = require('../models/articleModel');

// 1. Creer un nouvel utilisateur (POST /api/users)
const createUser = async (req, res) => {
  try {
    const { name, email, age, role, bio, phone } = req.body;
    if (!name || !email) {
      return res.status(400).json({ message: "Le nom et l'email sont requis"
          });
    }
    // Verification de doublon d'email
    const exists = await User.findOne({ email });
    if (exists) {
      return res.status(409).json({ message: 'Cet email est deja utilise' })
          ;
    }
    const user = await User.create({ name, email, age, role, bio, phone });
    res.status(201).json(user);
  } catch (err) {
    if (err.name === 'ValidationError') {
      return res.status(400).json({ message: err.message });
    }
    res.status(500).json({ message: err.message });
  }
};

// 2. Obtenir tous les utilisateurs (GET /api/users)
const getUsers = async (req, res) => {
  try {
    const { role } = req.query;
    const filter = role ? { role } : {};
    const users = await User.find(filter);
    res.status(200).json(users);
```

```
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

// 3. Obtenir un utilisateur par ID avec gestion CastError (GET /api/users/:id)
const getUserById = async (req, res) => {
  try {
    const user = await User.findById(req.params.id);
    if (!user) {
      return res.status(404).json({ message: 'Utilisateur non trouve' });
    }
    res.status(200).json(user);
  } catch (err) {
    // Exercice 2 : Interception CastError
    if (err.name === 'CastError') {
      return res.status(400).json({ message: 'Identifiant de ressource invalide (format ObjectId attendu)' });
    }
    res.status(500).json({ message: err.message });
  }
};

// 4. Supprimer un utilisateur avec controle d'integrite relationnelle (DELETE /api/users/:id)
const deleteUser = async (req, res) => {
  try {
    const user = await User.findById(req.params.id);
    if (!user) {
      return res.status(404).json({ message: 'Utilisateur non trouve' });
    }

    // Verification d'integrite relationnelle (Exercice 1)
    const hasArticles = await Article.findOne({ author: req.params.id });
    if (hasArticles) {
      return res.status(400).json({ message: "Impossible de supprimer un utilisateur ayant des articles publies" });
    }

    await User.findByIdAndDelete(req.params.id);
    res.status(200).json({ message: 'Utilisateur supprime avec succes', id: req.params.id });
  } catch (err) {
    // Exercice 2 : Interception CastError
    if (err.name === 'CastError') {
      return res.status(400).json({ message: 'Identifiant de ressource invalide (format ObjectId attendu)' });
    }
    res.status(500).json({ message: err.message });
  }
};

module.exports = { createUser, getUsers, getUserById, deleteUser };
```

3.7 Contrôleur Articles : src/controllers/articleController.js

```
const Article = require('../models/articleModel');
const User = require('../models/userModel');
```

```
3
4  // 1. Recuperer tous les articles avec l'auteur peuple via .populate()
5  const getAllArticles = async (req, res) => {
6    try {
7      const articles = await Article.find().populate('author', 'name email');
8      res.status(200).json(articles);
9    } catch (err) {
10     res.status(500).json({ message: err.message });
11   }
12 };
13
14 // 2. Creer un nouvel article relie a un auteur
15 const createArticle = async (req, res) => {
16   try {
17     const { title, content, authorId } = req.body;
18     if (!title || !content || !authorId) {
19       return res.status(400).json({ message: 'title, content et authorId
           sont obligatoires' });
20     }
21     // Verifier l'existence reelle de l'auteur
22     const author = await User.findById(authorId);
23     if (!author) {
24       return res.status(404).json({ message: 'Auteur non trouve (identifiant
            authorId invalide)' });
25     }
26     const newArticle = await Article.create({ title, content, author:
          authorId });
27     const populatedArticle = await newArticle.populate('author', 'name email
          ');
28     res.status(201).json(populatedArticle);
29   } catch (err) {
30     if (err.name === 'CastError') {
31       return res.status(400).json({ message: 'Identifiant de ressource
           invalide (format ObjectId attendu)' });
32     }
33     if (err.name === 'ValidationError') {
34       return res.status(400).json({ message: err.message });
35     }
36     res.status(500).json({ message: err.message });
37   }
38 };
39
40 // 3. Recuperer un article par ID (Exercice 2)
41 const getArticleById = async (req, res) => {
42   try {
43     const article = await Article.findById(req.params.id).populate('author',
          'name email');
44     if (!article) {
45       return res.status(404).json({ message: 'Article non trouve' });
46     }
47     res.status(200).json(article);
48   } catch (err) {
49     if (err.name === 'CastError') {
50       return res.status(400).json({ message: 'Identifiant de ressource
           invalide (format ObjectId attendu)' });
51     }
52     res.status(500).json({ message: err.message });
53   }
54 };
55
```

```
// 4. Mettre a jour un article
const updateArticle = async (req, res) => {
  try {
    const article = await Article.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true, runValidators: true }
    );
    if (!article) {
      return res.status(404).json({ message: 'Article introuvable pour la mise a jour' });
    }
    const populated = await article.populate('author', 'name email');
    res.status(200).json(populated);
  } catch (err) {
    if (err.name === 'CastError') {
      return res.status(400).json({ message: 'Identifiant de ressource invalide (format ObjectId attendu)' });
    }
    res.status(500).json({ message: err.message });
  }
};

// 5. Supprimer un article
const deleteArticle = async (req, res) => {
  try {
    const article = await Article.findByIdAndDelete(req.params.id);
    if (!article) {
      return res.status(404).json({ message: 'Article introuvable pour la suppression' });
    }
    res.status(200).json({ message: 'Article supprime avec succes', id: req.params.id });
  } catch (err) {
    if (err.name === 'CastError') {
      return res.status(400).json({ message: 'Identifiant de ressource invalide (format ObjectId attendu)' });
    }
    res.status(500).json({ message: err.message });
  }
};

module.exports = { getAllArticles, createArticle, getArticleById, updateArticle, deleteArticle };
```

3.8 Routeurs Express découplés : src/routes/userRoutes.js et src/routes/arti

```
// src/routes/userRoutes.js
const express = require('express');
const router = express.Router();
const userController = require('../controllers/userController');

router.get('/', userController.getUsers);
router.get('/:id', userController.getUserById);
router.post('/', userController.createUser);
router.delete('/:id', userController.deleteUser);

module.exports = router;
```

```
// src/routes/articleRoutes.js
const express = require('express');
const router = express.Router();
const articleController = require('../controllers/articleController');

router.get('/', articleController.getAllArticles);
router.get('/:id', articleController.getArticleById);
router.post('/', articleController.createArticle);
router.put('/:id', articleController.updateArticle);
router.delete('/:id', articleController.deleteArticle);

module.exports = router;
```

Conclusion

Ce TP 3 concrétise la puissance de l'écosystème MERN en combinant l'architecture modulaire SoC, Express, Mongoose et MongoDB. Les données sont validées en amont par les schémas Mongoose, les relations sont modélisées et peuplées élégamment avec `.populate()`, et les erreurs de cast et d'intégrité sont gérées avec professionnalisme. L'API est robuste, extensible et prête pour un déploiement en production.